WEEK-9- Implementation of Elastic Load Balancer with Auto Scaling using AWS EC2 Instances.

Elastic Load Balancer (ALB)

Step 1: Launch Two EC2 Instances

1.1 Launch EC2 Instance #1

1. Go to **AWS EC2 Console → Launch Instance**.
2. **Name**: webserver-1
3. Choose an **AMI** (Amazon Linux 2).
4. Select **Instance Type** (e.g., t2.micro).
5. Add **Storage** (default is fine).
6. Configure **Security Group**:
    - Allow **SSH (22)** from your IP
    - Allow **HTTP (80)** from anywhere (0.0.0.0/0)
7. **Launch** → Select/Create Key Pair → Launch.

    Run these commands after connecting VM

    ```
    yum update -y
    yum install httpd -y
    systemctl start httpd
    systemctl enable httpd

    echo  "This is Server 1" > /var/www/html/index.html
    ```

✅ Now you create second **EC2 instances** running Apache, with message **This is Servier 2** each serving a simple web page.

Repeat it for second instance

```
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd
```

```
echo  "This is Server 2" > /var/www/html/index.html
```

Step 2: Create a Load Balancer

AWS provides **Elastic Load Balancer (ELB)**. We will create an **Application Load Balancer (ALB)**.

2.1 Configure Security Group for Load Balancer

- Go to **EC2 → Security Groups → Create Security Group**
    - Name: lb-sg
    - Allow **HTTP (80)** from anywhere
    - Optional: HTTPS (443) if using SSL

2.2 Create the Load Balancer

1. Go to **EC2 → Load Balancers → Create Load Balancer → Application Load Balancer**
2. **Basic Configuration**:
    - Name: my-alb
    - Scheme: Internet-facing
    - IP address type: IPv4
3. **Listeners**:
    - Default: HTTP (80)

2.3 Configure Availability Zones

- Select the deafault **VPC** where your EC2 instances exist
- Select **two /all subnets** (one for each instance if possible)

2.4 Configure Security Groups

- Choose the **lb-sg** security group created earlier

2.5 Configure Target Groups

1. **Create a new target group**:
    - Name: web-servers-tg
    - Target type: Instance
    - Protocol: HTTP
    - Port: 80
    - Health checks: / (default)
2. **Register Targets**:
    - Select EC2 INSTANCES HERE --- **webserver-1** and **webserver-2**
    - Click **Include as pending** → **Register**

2.6 Review and Create

- Review all settings → Click **Create Load Balancer**
- AWS will take a few minutes to provision the ALB

Step 3: Verify Load Balancer

1. Go to **Load Balancers → Select ALB → Description → DNS name**
2. Copy the **DNS name** (something like my-alb-123456.us-east-1.elb.amazonaws.com)
3. Paste it in a browser → You should see either **WebServer-1** or **WebServer-2** page.
    - Refresh multiple times → Load balancer distributes traffic between the two instances

Step 4: Optional – Test Health Checks

- Go to **Target Groups → web-servers-tg → Targets**
- Ensure **Status** of both instances is healthy

If a server is unhealthy, check the bootstrapping script or security group rules.

Step-by-Step: Auto Scaling Group with Load Balancer

Step 1: Create an AMI from Existing EC2 Instance

1. Go to **EC2 Console → Instances**
2. Select the EC2 instance you already configured with application
3. Click **Actions → Image and Templates → Create Image**
4. **Provide Image Name**: e.g., my-webserver-AMI
5. Optional: Add description
6. Click **Create Image**
7. Wait for the AMI to become **available** (status: available in **AMIs** section)

✔ This AMI will be used for launching new instances automatically in ASG.

Step 2: Create a Launch Template

1. Go to **EC2 → instances->Launch Templates → Create Launch Template**
2. **Launch Template Name**: my-launch-template
3. **AMI**: Select the AMI created in Step 1 (my-webserver-AMI)
4. **Instance Type**: t2.micro (or your choice)
5. **Key Pair**: Select your existing key pair
6. **Security Groups**: Select the **Load Balancer Security Group** (allow HTTP 80)
7. **User Data**: Optional (already baked into AMI)
8. Click **Create Launch Template**

✔ Launch Template is ready for Auto Scaling.

Step 3: Create Auto Scaling Group (ASG)

1. Go to **EC2 → Auto Scaling Groups → Create Auto Scaling Group**
2. **Name**: webserver-asg
3. **Launch Template**: Select my-launch-template
4. **Template Version**: Select **Version 1**
5. Click **Next**

Step 3.1: Network Settings

1. **VPC**: Default VPC (or your existing VPC)
2. **Availability Zones (AZs)**: Select **all** subnets (for high availability)
3. **Load Balancer**: Enable **Attach to an existing load balancer**
    - Select your **existing Load Balancer**
4. **Health Check**: HTTP
    - Set **Health Check Grace Period**: 300 seconds
5. Click **Next**

Step 3.2: Configure Group Size

1. **Desired Capacity**: 2 (initial instances)
2. **Minimum Capacity**: 1
3. **Maximum Capacity**: 4
4. Click **Next**

Step 3.3: Review and Create ASG

1. Review all settings
2. Click **Create Auto Scaling Group**

✔ Auto Scaling Group is now ready and attached to your Load Balancer.

Step 4: Verify Load Balancer and Auto Scaling

1. Go to **Load Balancers → Select your Load Balancer**
2. Click **Instances → Target Group → Registered Targets**
    - You will see **2 healthy instances** initially
3. Check **Auto Scaling Group → Instances**
    - You may see up to **4 instances** if scaling triggers
4. Test **auto recovery**:
    - **Terminate one of the instances**
    - After a few seconds, **ASG will launch a new instance automatically**
    - Traffic will automatically be **distributed by the Load Balancer**

Step 5: Test the Load Balancer

1. Copy the **Load Balancer DNS Name** from ALB
2. Open in a browser → You should see your web page from one of the instances
3. Refresh multiple times → traffic alternates between instances

✅ This confirms **Auto Scaling + Load Balancer** is working correctly.

Summary of Flow

1. **Create AMI** → snapshot of working EC2
2. **Create Launch Template** → using AMI, security group
3. **Create Auto Scaling Group** → attach Launch Template, set min/desired/max, attach Load Balancer
4. **Verify instances** → ASG launches/replaces instances automatically
5. **Test Load Balancer** → distributes traffic between all active instances

Scenario Based Questions (ELB & Auto Scaling)

1) A company hosts a web application on two EC2 instances, but sometimes one server becomes overloaded while the other remains idle.
   Which AWS service can distribute incoming traffic evenly across both servers?
2) A developer created multiple EC2 instances for a web application but wants users to access the application using a single entry point instead of multiple server IP addresses.
   Which AWS service can solve this problem?
3) An organization notices that during peak hours their application receives heavy traffic and servers become slow. During non-peak hours, many servers remain unused.
   Which AWS feature can automatically increase or decrease the number of servers based on demand?
4) A company deployed a load balancer but users are still getting errors because traffic is being sent to an unhealthy server.
   Which load balancer feature helps detect and stop sending traffic to failed servers?
5) A developer wants to automatically launch additional EC2 instances when CPU utilization exceeds 70%.
   Which AWS service configuration should be used to implement this requirement?
6) An application receives traffic from users worldwide, and the company wants to ensure that requests are distributed across multiple EC2 instances for high availability.
   How can Elastic Load Balancer help in this architecture?
7) A developer configured an Auto Scaling Group, but new instances are not launching even when traffic increases.
   What configuration might be missing or incorrect?
8) A company wants to maintain a minimum of two running servers at all times, but automatically add more servers when traffic increases.
   Which Auto Scaling configuration parameters should be defined?
9) A web application running on EC2 instances behind a load balancer suddenly crashes when traffic spikes.
   How can combining Elastic Load Balancer with Auto Scaling improve system reliability?
10) A developer created an Auto Scaling Group but forgot to attach it to a load balancer.
    What potential problem might occur in handling user requests?
11) A company wants to replace unhealthy EC2 instances automatically to maintain application availability.
    Which AWS feature can automatically detect and replace failed instances?
12) A developer wants to monitor the performance of EC2 instances and trigger scaling actions based on metrics such as CPU usage or network traffic.
    Which AWS service can provide these monitoring metrics?
13) A startup expects sudden spikes in website traffic during a promotional event and wants the infrastructure to automatically handle the load.
    How can Auto Scaling help manage this situation?
14) A developer wants to ensure that incoming user requests are routed to servers based on HTTP or HTTPS protocols.
    Which type of AWS load balancer should be used?

15) A company wants to build a highly available architecture where traffic is automatically balanced across multiple servers and new servers are launched when demand increases.
Which AWS services should be combined to achieve this solution?